

# **CS111**

Introduction to Computing Science

# Using Arrays

- Arrays are **data structures**.
  - They are used to store a group of objects/ variables.
- All elements of an array must be of the **same data**
  - type: *int, float, char, double*.
- The elements are stored **sequentially in memory**
  - This allows for 'easy' access.
  - It also allow for easy mistakes.

# Array Syntax

## Defining an Array

Element type    Name    Size

```
double values[5] = { 32, 54, 67.5, 29, 34.5 };
```

Size must be a constant.

Ok to omit size if initial values are given.

Use brackets to access an element.

```
values[i] = 0;
```

Optional list of initial values

The index must be  $\geq 0$  and  $<$  the size of the array.

# Common Array and Vector Algorithms

- There are many typical things that are done with sequences of values.
- There many common algorithms for processing values stored in arrays.
  - Filling
  - Maximum and minimum
  - Sum and average
  - Copying
  - ...

# Common Algorithms – Filling

This loop fills an array of size N with zeros:

```
for (int i = 0; i < N; i++)  
{  
    values[i] = 0;  
}
```

# Common Algorithms – Filling

Here, we fill the array with squares (0, 1, 4, 9, 16, ...).

Note that the element with index 0 will contain  $0^2$ , the element with index 1 will contain  $1^2$ , and so on.

```
for (int i = 0; i < N; i++)  
{  
    squares[i] = i * i;  
}
```

# Common Algorithms – Copying

Consider these two arrays:

```
int squares[5] = { 0, 1, 4, 9, 16 };  
int lucky_numbers[5];
```

How can we copy the values  
from `squares`  
to `lucky_numbers`?

# Common Algorithms – Copying

Let's try what seems right and easy...

```
squares = lucky_numbers;
```

...and wrong!

You cannot assign arrays!

You will have to do your own work,.

# Common Algorithms – Copying

```
int squares[5] = { 0, 1, 4, 9, 16 };  
int lucky_numbers[5];
```

```
for (int i = 0; i < 5; i++)  
{  
    lucky_numbers[i] = squares[i];  
}
```

when i is 0

|           |    |     |
|-----------|----|-----|
| squares = | 0  | [0] |
|           | 1  | [1] |
|           | 4  | [2] |
|           | 9  | [3] |
|           | 16 | [4] |

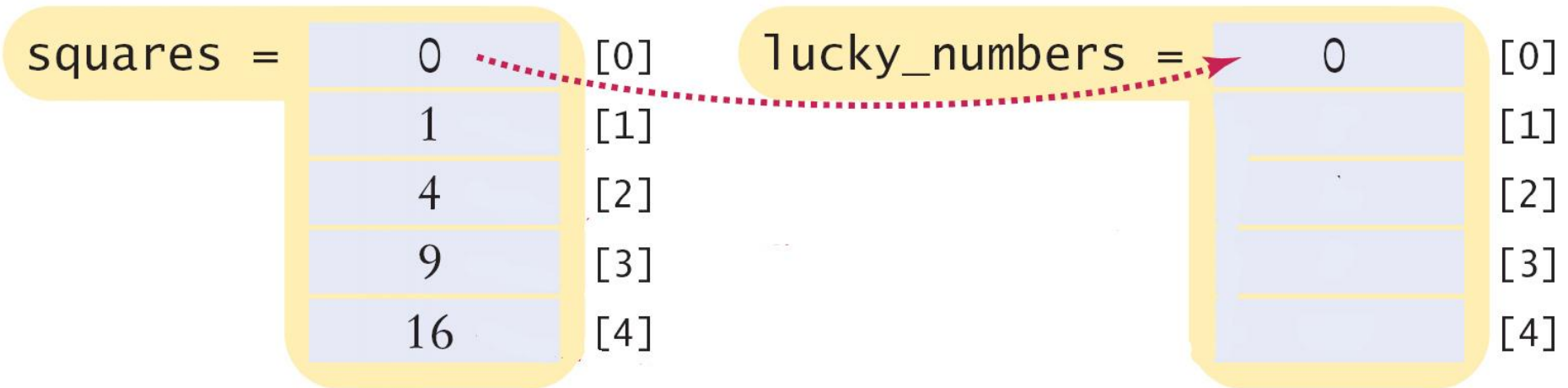
|                 |  |     |
|-----------------|--|-----|
| lucky_numbers = |  | [0] |
|                 |  | [1] |
|                 |  | [2] |
|                 |  | [3] |
|                 |  | [4] |

# Common Algorithms – Copying

```
int squares[5] = { 0, 1, 4, 9, 16 };  
int lucky_numbers[5];
```

```
for (int i = 0; i < 5; i++)  
{  
    lucky_numbers[i] = squares[i];  
}
```

when i is 0

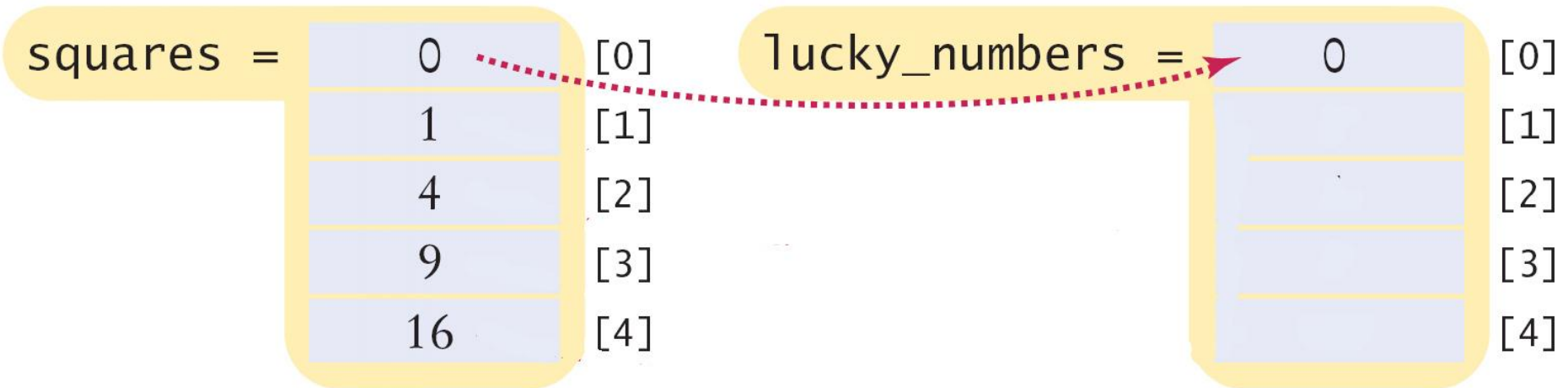


# Common Algorithms – Copying

```
int squares[5] = { 0, 1, 4, 9, 16 };  
int lucky_numbers[5];
```

```
for (int i = 0; i < 5; i++)  
{  
    lucky_numbers[i] = squares[i];  
}
```

when i is 1

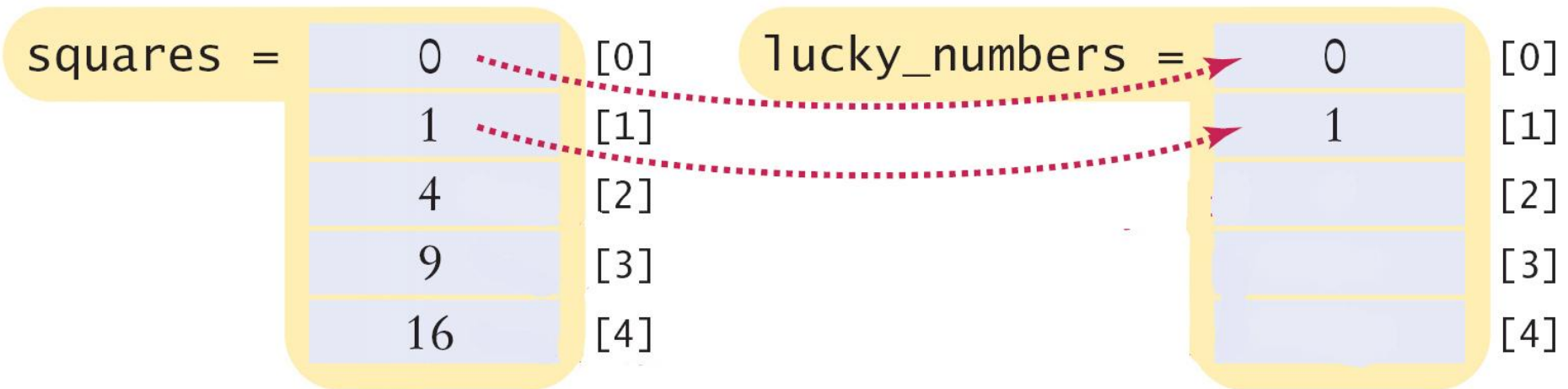


# Common Algorithms – Copying

```
int squares[5] = { 0, 1, 4, 9, 16 };  
int lucky_numbers[5];
```

```
for (int i = 0; i < 5; i++)  
{  
    lucky_numbers[i] = squares[i];  
}
```

when i is 1

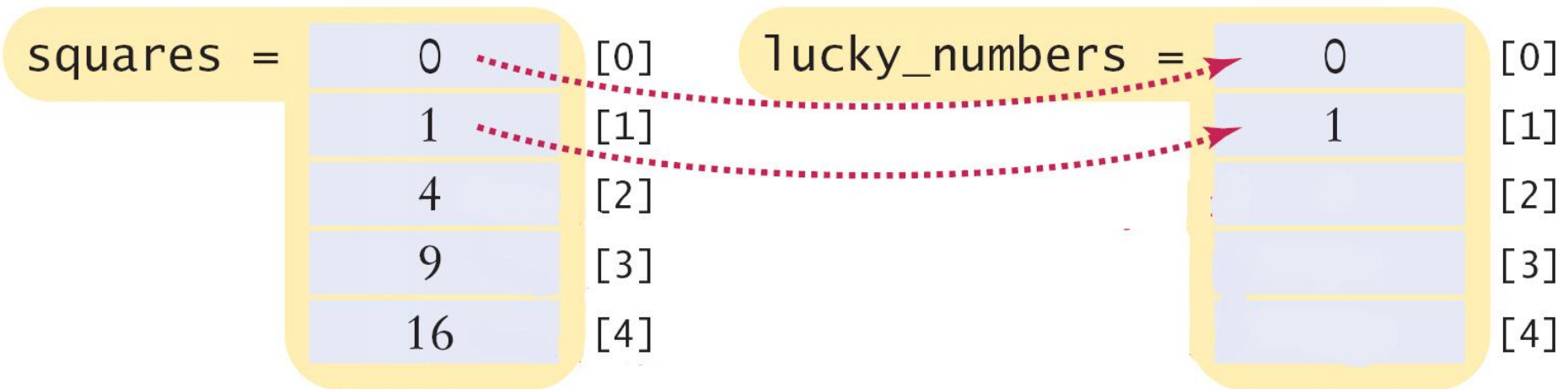


# Common Algorithms – Copying

```
int squares[5] = { 0, 1, 4, 9, 16 };  
int lucky_numbers[5];
```

```
for (int i = 0; i < 5; i++)  
{  
    lucky_numbers[i] = squares[i];  
}
```

when i is 2

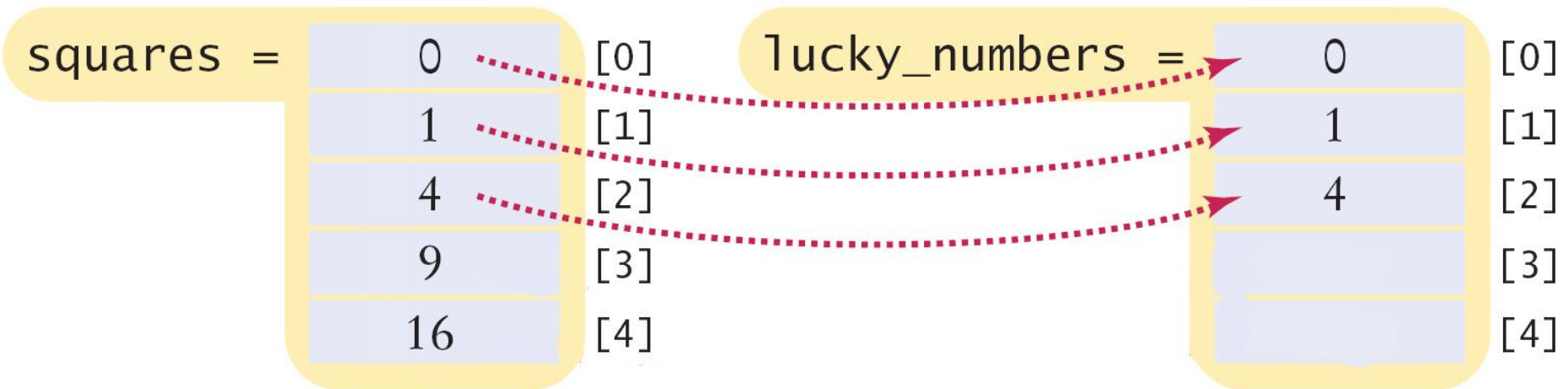


# Common Algorithms – Copying

```
int squares[5] = { 0, 1, 4, 9, 16 };  
int lucky_numbers[5];
```

```
for (int i = 0; i < 5; i++)  
{  
    lucky_numbers[i] = squares[i];  
}
```

when i is 2

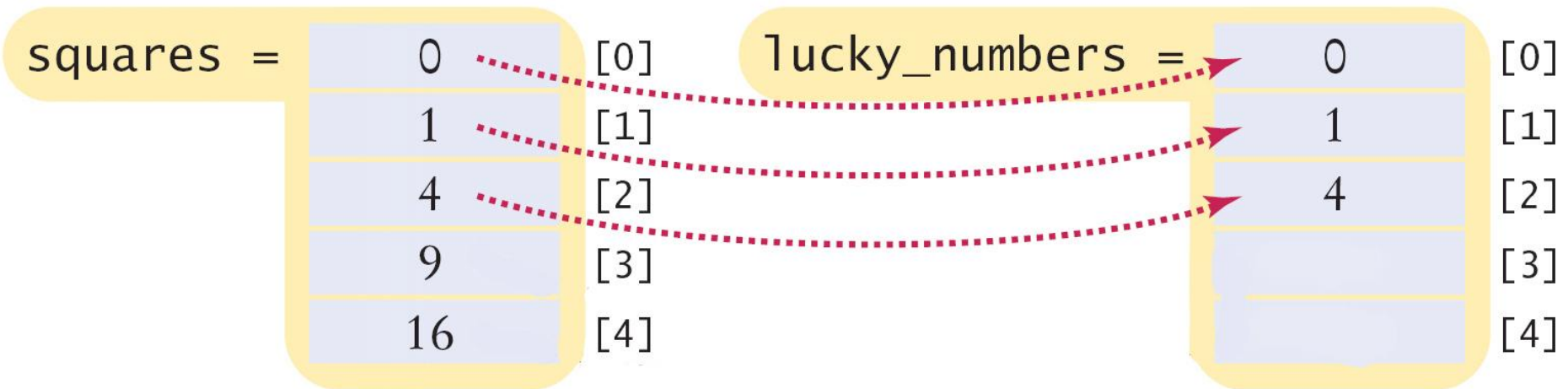


# Common Algorithms – Copying

```
int squares[5] = { 0, 1, 4, 9, 16 };  
int lucky_numbers[5];
```

```
for (int i = 0; i < 5; i++)  
{  
    lucky_numbers[i] = squares[i];  
}
```

when i is 3

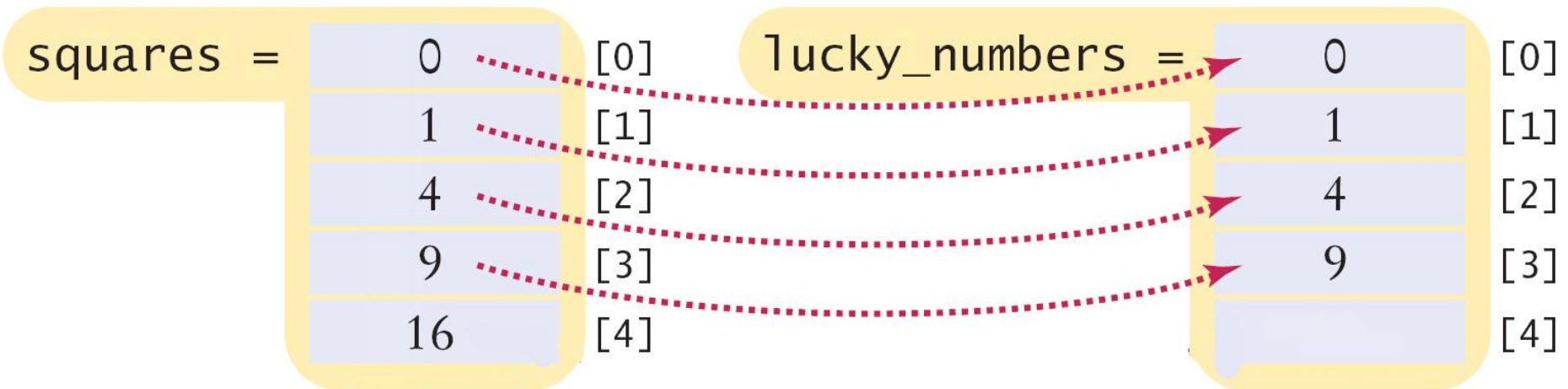


# Common Algorithms – Copying

```
int squares[5] = { 0, 1, 4, 9, 16 };  
int lucky_numbers[5];
```

```
for (int i = 0; i < 5; i++)  
{  
    lucky_numbers[i] = squares[i];  
}
```

when i is 3

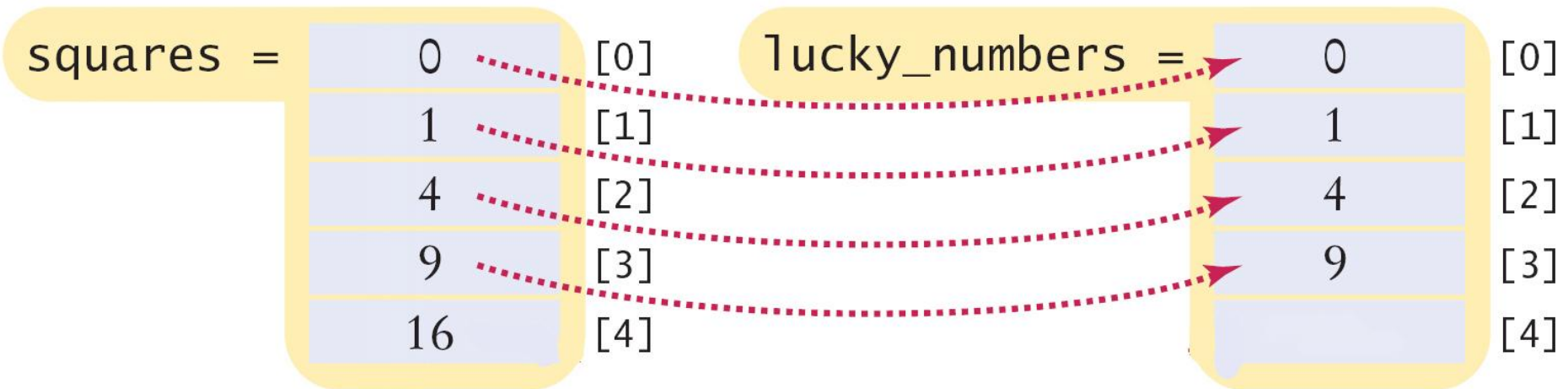


# Common Algorithms – Copying

```
int squares[5] = { 0, 1, 4, 9, 16 };  
int lucky_numbers[5];
```

```
for (int i = 0; i < 5; i++)  
{  
    lucky_numbers[i] = squares[i];  
}
```

when i is 4

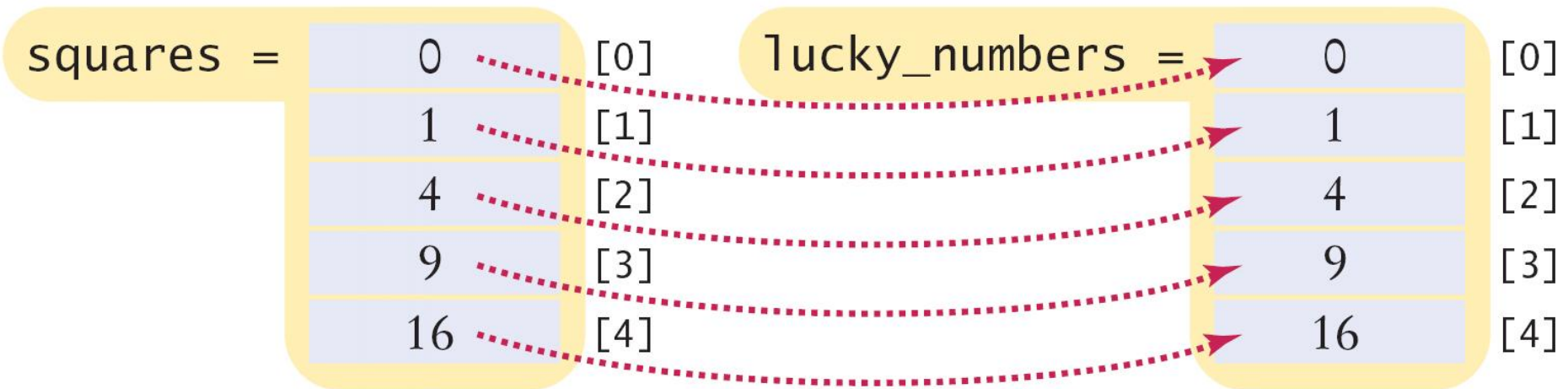


# Common Algorithms – Copying

```
int squares[5] = { 0, 1, 4, 9, 16 };  
int lucky_numbers[5];
```

```
for (int i = 0; i < 5; i++)  
{  
    lucky_numbers[i] = squares[i];  
}
```

when i is 4



# Common Algorithms – Sum and Average Value

You have already seen the algorithm for computing the sum and average of set of data.

The algorithm is the same when the data is stored in an array

```
double total = 0;
for (int i = 0; i < N; i++)
{
    total = total + values[i];
}
```

The average is just arithmetic:

```
double average = total / N;
```

# Common Algorithms – Who Is the Tallest?



Who's the tallest in the line?

# Common Algorithms – Who Is the Tallest?

If everyone's height is stored in an array,  
determining the largest value  
(who's the tallest person's height?)  
is just another algorithm...



# Common Algorithms – Maximum and Minimum

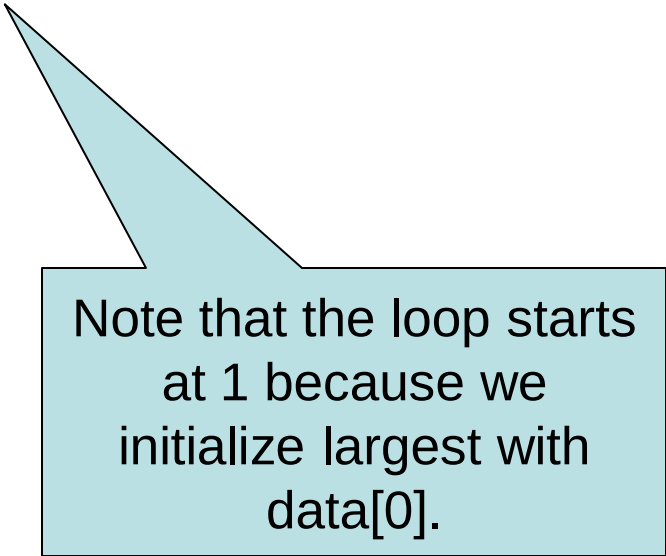
To compute the largest value in a vector, keep a variable that stores the largest element that you have encountered, and update it when you find a larger one.

```
double largest = values[0];
for (int i = 1; i < N; i++)
{
    if (values[i] > largest)
    {
        largest = values[i];
    }
}
```

# Common Algorithms – Maximum and Minimum

To compute the largest value in a vector, keep a variable that stores the largest element that you have encountered, and update it when you find a larger one.

```
double largest = values[0];  
for (int i = 1; i < N; i++)  
{  
    if (values[i] > largest)  
    {  
        largest = values[i];  
    }  
}
```



Note that the loop starts at 1 because we initialize largest with data[0].

# Common Algorithms – Who Is the Shortest?



Who's the shortest in the line?

# Common Algorithms – Maximum and Minimum

For the minimum, we just reverse the comparison.

```
double smallest = values[0];  
for (int i = 1; i < size of values; i++)  
{  
    if (values[i] < smallest)  
    {  
        smallest = values[i];  
    }  
}
```

These algorithms require that the array contain at least one element.

# Common Algorithms – Element Separators

When you display the elements of a vector, you usually want to separate them, often with commas or vertical lines, like this:

1 | 4 | 9 | 16 | 25

Note that there is one fewer separator than there are numbers.

To print five elements, you need *four* separators.



# Common Algorithms – Element Separators

Print the separator before each element  
*except the initial one* (with index 0):

1 | 4 | 9 | 16 | 25

```
for (int i = 0; i < size of values; i++)  
{  
    if (i > 0)  
    {  
        cout << " | ";  
    }  
    cout << values[i];  
}
```



# Common Algorithms – Element Separators

Sometimes you need to print separators on all sides

| 1 | 4 | 9 | 16 | 25 |



```
cout << "|";  
for (int i = 0; i < size of values; i++)  
{  
  
    cout << values[i] << "|";  
}
```



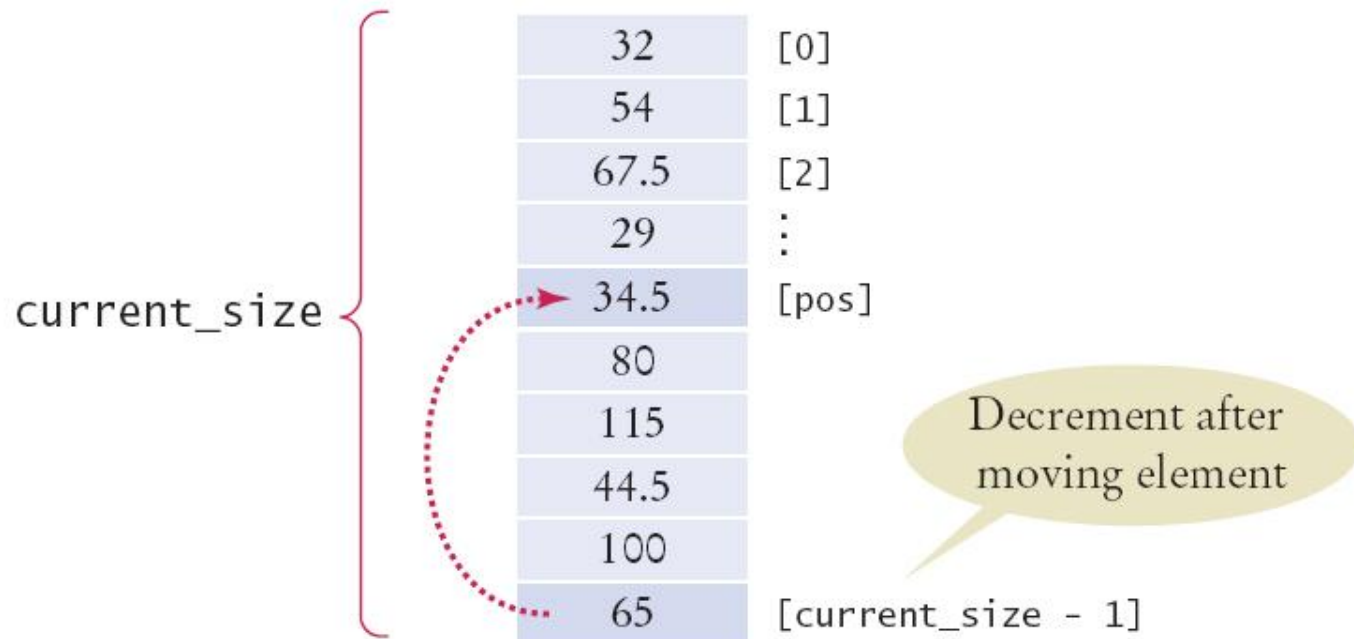
# Common Algorithms – Linear Search

Find the position of a certain value, say 100, in an array:

```
int pos = 0;
bool found = false;
while (pos < N && !found)
{
    if (values[pos] == 100) // looking for 100
    {
        found = true;
    }
    else
    {
        pos++;
    }
}
```

# Common Algorithms – Removing an Element, Unordered

- Suppose you want to remove the element at index  $i$ .
- If the elements in the vector are not in any particular order, that task is easy to accomplish.
- Simply overwrite the element to be removed with the *last* element of the vector, then shrink the size of the vector by removing the value that was copied.



# Common Algorithms – Removing an Element, Ordered

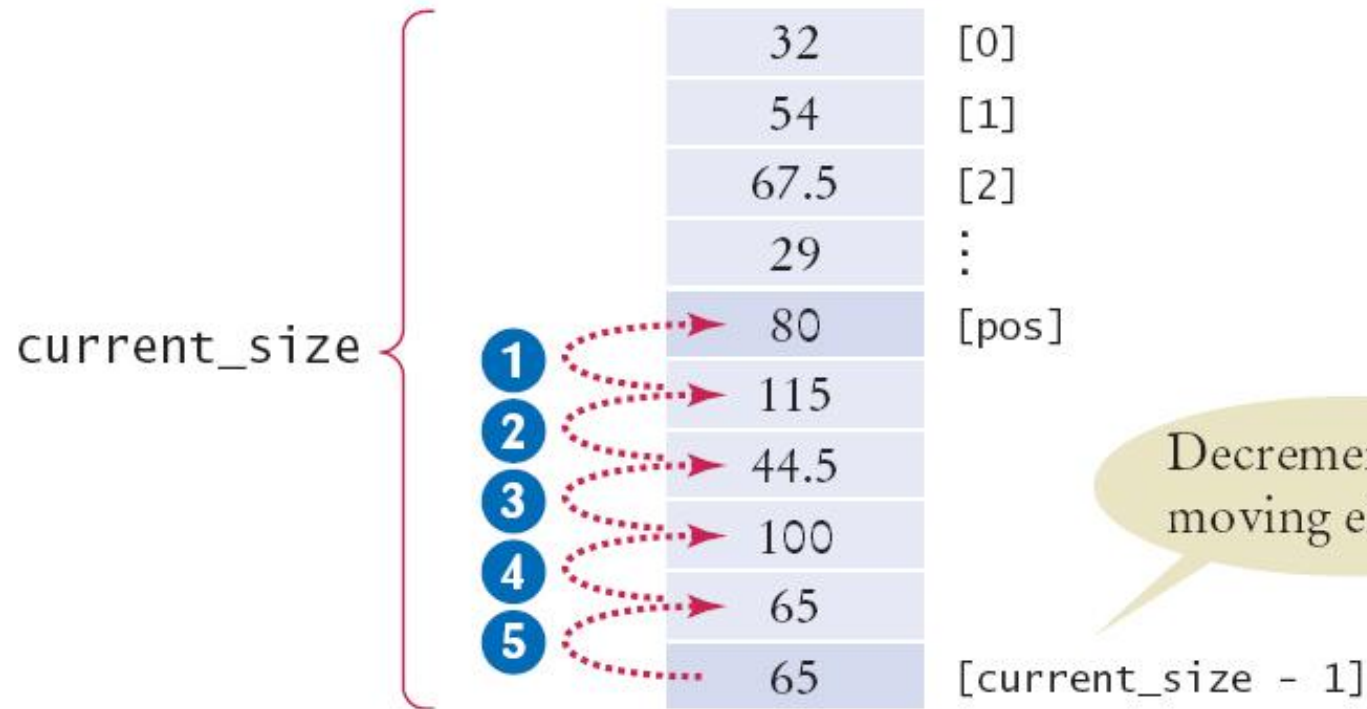
The situation is more complex  
if the order of the elements matters.

Then you must move all elements following the element to be removed “down” (to a lower index), and then shrink the size of the vector by removing the last element.

```
for (int i = pos + 1; i < current_size; i++)  
{  
    values[i - 1] = values[i];  
}  
current_size--;
```

# Common Algorithms – Removing an Element, Ordered

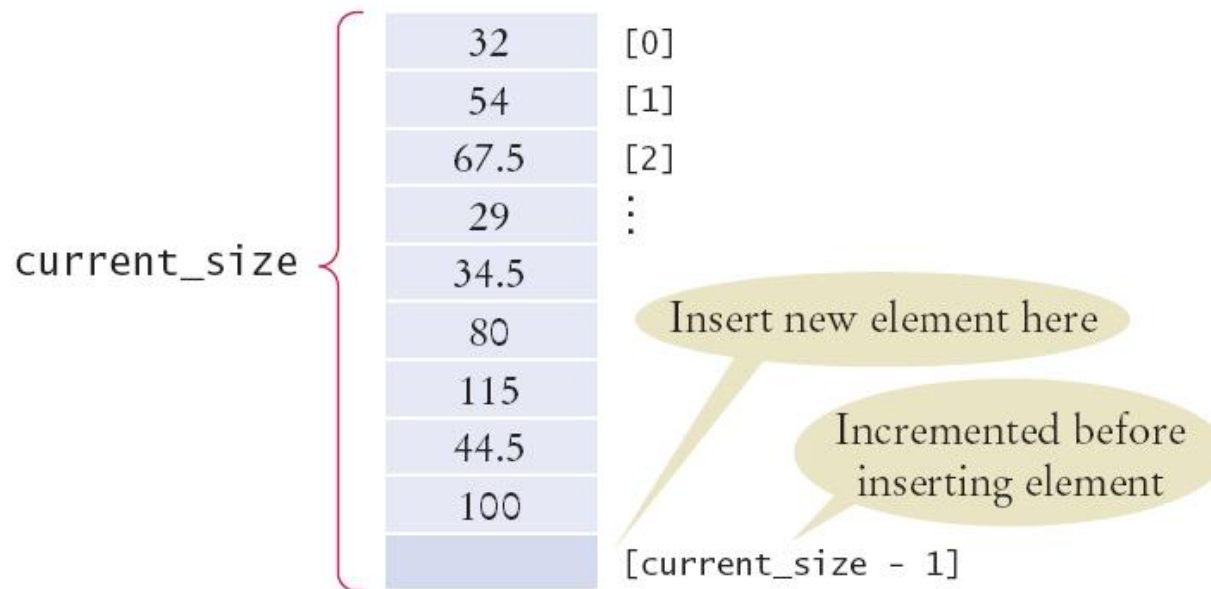
```
for (int i = pos + 1; i < current_size; i++)  
{  
    values[i - 1] = values[i];  
}  
current_size--;
```



# Common Algorithms – Inserting an Element Unordered

If the order of the elements does not matter, in a partially filled array (which is the only kind you can insert into), you can simply insert a new element at the end.

```
if (current_size < CAPACITY)
{
    current_size++;
    values[current_size - 1] = new_element;
}
```

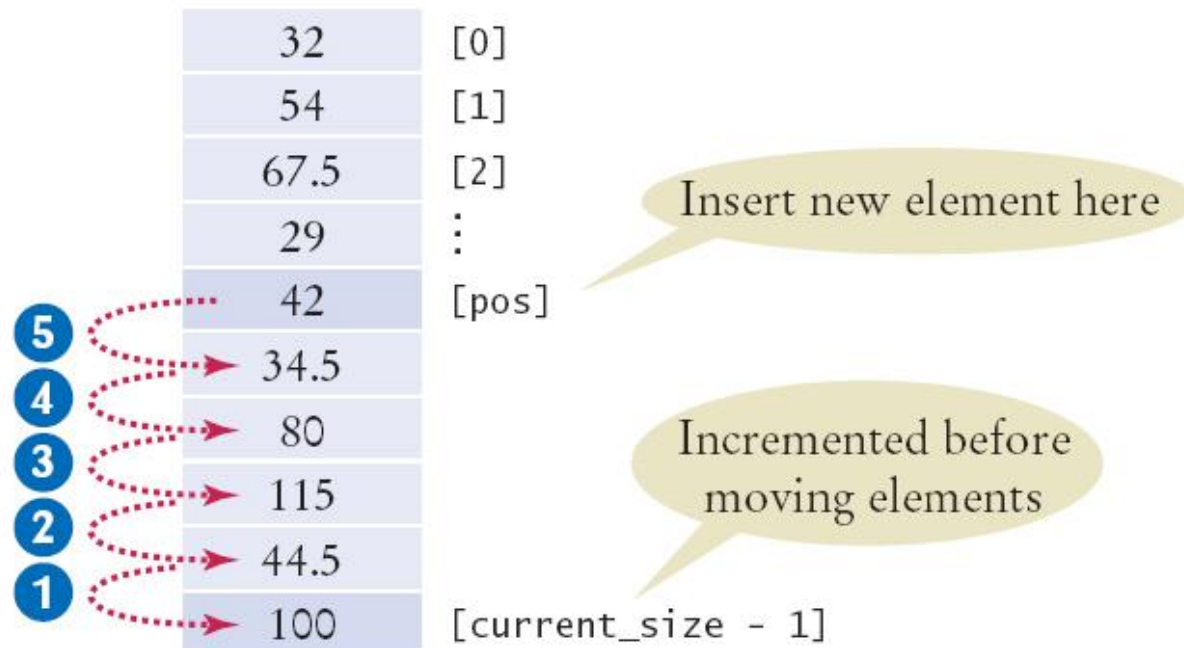


# Common Algorithms – Inserting an Element Ordered

If the order of the elements *does* matter, it is a bit harder.

To insert an element at position  $i$ , all elements from that location to the end of the vector must be moved “up”.

After that, insert the new element at the now vacant position  $[i]$ .



# Common Algorithms – Inserting an Element Ordered

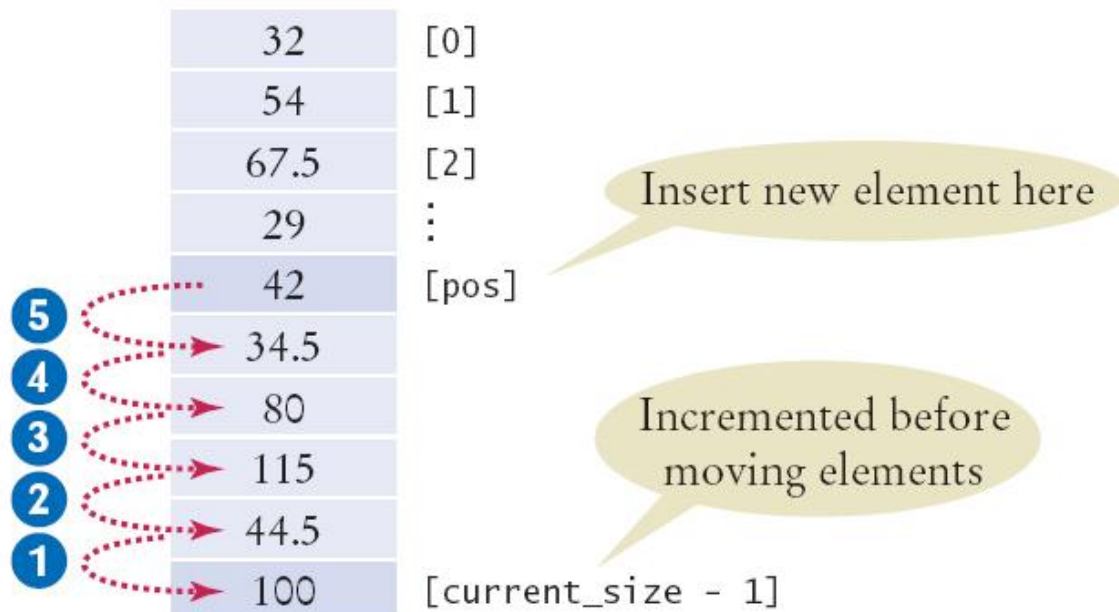
First, you must make the array one larger by incrementing `current_size`.

Next, move all elements above the insertion location to a higher index.

Finally, insert the new element in the place you made for it.

# Common Algorithms – Inserting an Element Ordered

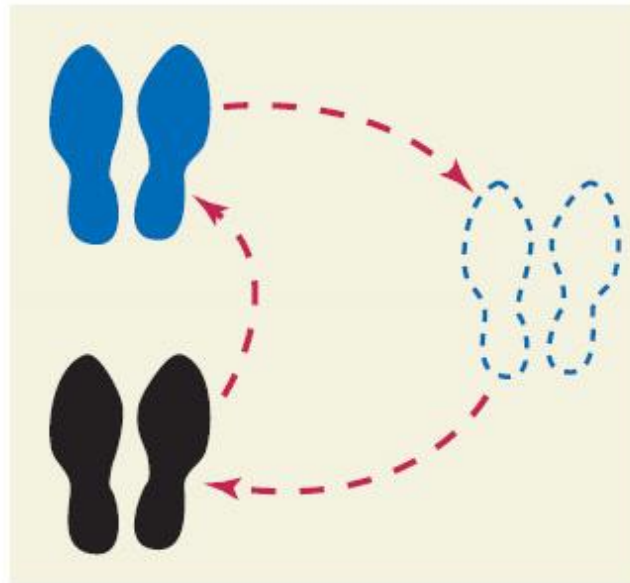
```
if (current_size < CAPACITY)
{
    current_size++;
    for (int i = current_size - 1; i > pos; i--)
    {
        values[i] = values[i - 1];
    }
    values[pos] = new_element;
}
```



# Common Algorithms – Swapping Elements

Swapping two elements in an array is an important part of sorting an array.

To do a swap of two things, you need *three* things!



# Common Algorithms – Swapping Elements

Suppose we need to swap the values at positions *i* and *j* in the array.

Will this work?

```
values[i] = values[j];  
values[j] = values[i];
```

Look closely!

In the first line you lost – forever! – the value at *i*, replacing it with the value at *j*.

Then what?

Put *j*'s value back in *j* in the second line?

# Common Algorithms – Swapping Elements

This is better

```
temp = values[i];  
values[i] = values[j];  
values[j] = temp;
```

- Store one value in a temporary location
- Over write that value with the second
- Overwrite the second with the value in the temp

# Common Algorithms – Reading Input

If the know how many input values the user will supply, you can store them directly into the array:

```
double values[NUMBER_OF_INPUTS];  
for (i = 0; i < NUMBER_OF_INPUTS; i++)  
{  
    cin >> values[i];  
}
```

# Common Algorithms – Reading Input

When there will be an arbitrary number of inputs, things get more complicated.

- Add values to the end of the array until all inputs have been made.
- Again, the companion variable will have the number of inputs.

```
double values[CAPACITY];
int current_size = 0;
double input;
while (cin >> input)
{
    if (current_size < CAPACITY)
    {
        values[current_size] = input;
        current_size++;
    }
}
```

# Common Algorithms – Reading Input

Unfortunately it's even more complicated:

Once the array is full, we allow the user to keep entering!

Because we can't change the size of an array after it has been created, we'll just have to give up for now.